Baker College | GREATNESS IS EARNED HERE.

PLC Programming Instructions

Programmable Logic Controllers

Brad Peirson
College of Information Technology and Engineering

Agenda

PLC Data Types

Basic PLC Programming Instructions

PLC Timers and Counters

Advanced PLC Programming Instructions

Data Manupulation Instructions

Student Learning Outcomes

1. Create programs in Ladder Logic using the following instructions in an application
2. Program PLC timers and counters

PLC Data Types

BIT

- 1 bit
- The smallest unit of memory
- Represents discrete binary states
    - 0 (OFF/FALSE)
    - 1 (ON/TRUE)

NIBBLE

- 4 bits
- 1 binary coded decimal (BCD) digit

BYTE

- 8 Bits
- 256 unique combinations
- Typically used for ASCII characters–1 BYTE = 1 character

WORD

- 16 Bits
- Historical standard register size in PLCs (AB SLC 500/Siemens S7-300)

DWORD/LWORD

- DWORD–32 bits
- LWORD–64 bits
- Modern architecture
- Used for high precision motion/math in older systems

Sub-Bit Access

- These data types are most often used as a number
- But they are *also* just a collection of bits
- We can access the individual bit inside a word using *dot notation*
  - `Word_Var.0`, `Word_Var.7`, etc.

IEC 61131-3 Elementary Types

Type	Keyword	Bits	Signed Range	Common Industrial Usage
Boolean	`BOOL`	1	0 or 1 (FALSE / TRUE)	Proximity sensors, pushbuttons, coils
Short Integer	`SINT`	8	-128 to +127	Small part counts, status flags
Integer	`INT`	16	-32,768 to +32,767	Analog inputs (0-10V, 4-20mA raw counts)
Double Integer	`DINT`	32	-2,147,483,648 to +2,147,483,647	High-speed encoders, totalizer counts
Real (Float)	`REAL`	32	IEEE 754 Floating Point	Scaled engineering units (PSI, GPM, °C)

Signed Integers

- Integers can be signed (+/-) or unsigned
- The defautl in most PLCs is signed–unsigned is optional
- In a signed integer the first bit is the sign: 0 = +, 1 = -
- Because we lose a bit, the capacity of the number is effectively cut in half
- `INT`: -32,768 to 32,767
- `UINT`: 0 to 65,535

Floating Point Numbers

- IEEE 754 Format
- 32-bits: 1 sign, 8 exponent bits, 23 mantissa bits
- An integer is *only* whole numbers, `REAL` must be used if a decimal point is necessary
- *NEVER* use an equality check (`REAL_1 = 10.0`) on a `REAL`–floating point numbers have in-built rounding errors

Addressing Memory in PLCs

- **Direct Memory Addressing:** Logic references explicit *physical* register locations in the CPU
- **Tag-Based Addressing:** Programmer creates descriptive, symbolic names stored in a database
- IEC 61131-3 standardizes both approaches
- Older PLCs pretty exclusively direct addressed, modern PLCs mostly use tag-based databases

Direct vs. Tag-Based Syntax

AB Direct (SLC/Micrologix)
Uses fixed file letters & numbers
`I:1/0`–Input slot 1, bit 0
`B3:0/0`–Binary bit word 0, bit 3
`N7:0`–Integer file 7, register 0

IEC 61131-3 Direct Syntax
Uses standard prefix notation
`%IX0.0`–Input byte 0, bit 0
`%QX0.0`–Output byte 0, bit 0
`%MW100`–Memory word 100

Tag-Based (Logix/TIA)
Uses symbolic variable names
`Conveyor_Start_PB`
`Tank_Level_Scaled`
Hardware mapped vis I/O aliasing

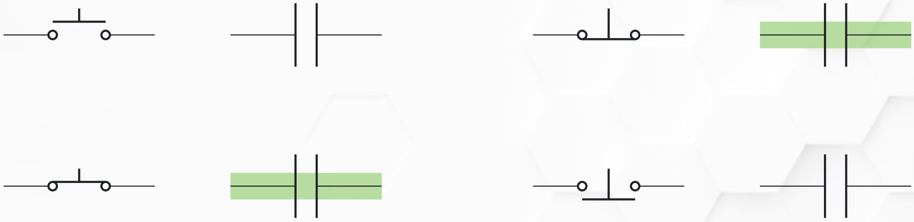
Basic PLC Programming Instructions

The 3 Basic Instructions

- Ladder logic doesn't care what the input *is*
- All that matters is on/off
- Remember, these were intentionally designed to look like relay contacts and coils

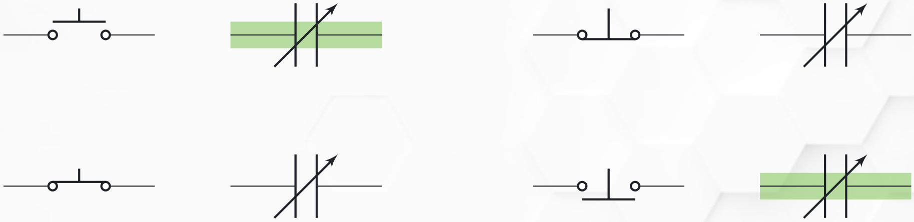
Examine if Closed (XIC)

True when the input is true (on)



Examine if Open (XIO)

True when the input is false (off)

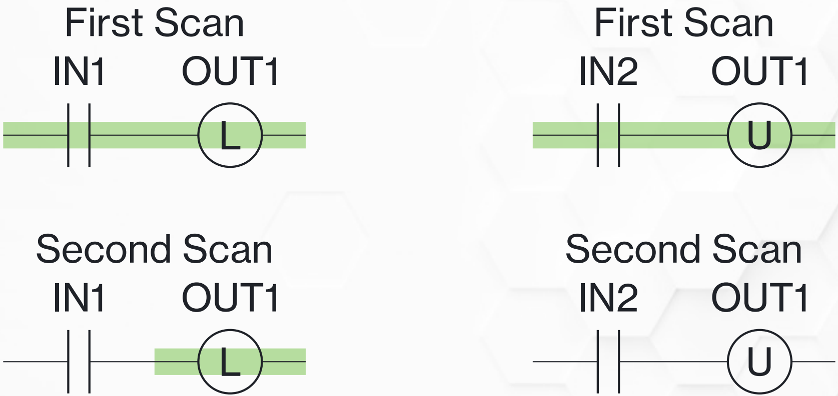


Output Energize (OTE)

True when the logic before it is true

Output Latch (OTL) and Unlatch (OTU)

Once turned on the outputs *stays on* until explcitly unlatched



Take Caution with Latches

- Be *very* careful with latches–my recommendation is to not use them as much as possible
- OTE is *volatile*–when power cycles the output is set to 0 (off)
- OTL is *NOT* volatile–if it's on when power goes out it will be on when power is restored
- This can and has caused unwanted/unsafe machine motion

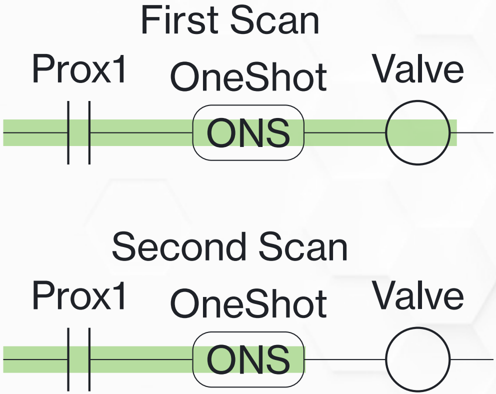
Latch/Unlatch on Key Platforms

How Key Platforms Handle Latch/Unlatch on Power Cycle

Manufacturer	Equivalent Instructions	Power Cycle Behavior	Memory Control Mehcanism
Allen-Bradley (Logix5000)	OTL/OTU	Retains state across power cycles (non-volatile)	Tied directly to the instruction choice (OTL/OTU memory retention vs. OTE non-retention)
Siemens (TIA Portal)	S (Set)/ R (Reset)	Resets to `0` unless declared as Retentive	Bit/DB retain settings in tag table
Beckhoff (TwinCAT)/ CODESYS	S (Set)/ R (Reset)	Resets to `0` unless located in Retain memory	`RETAIN` or `PERSISTENT` keywords
Mitsubishi (GX Works)	SET/RST	Retains state if mapped to Latch Range	Latch (L) device memory range
Omron (Sysmac Studio)	Set/Rst	Resets to `0` unless declared as Retentive	Retain attribute checkbox on VAR

Oneshot

A oneshot is true for *one scan* while the logic before it is true–the logic has to turn false to reset it



PLC Timers and Counters

Timers and Counters

- Timers time, counters count–but their under-the-hood structure is *very* similar
- “Timer” and “counter” refer to the *variable type*
- The timer or counter variable is then used in conjunction with a timer or counter instruction

Timer and Counter Variable Types

- Timers and counters are made up of 3 words:
  1. Word 0: Control word–used as bits, what the bits *do* vary between timers and counters
  2. Word 1: Preset (PRE)–the time/count target
  3. Word 2: Accumulated (ACC)–the current time/count

Timer Variables

EN Rung logic before the instruction is true
TT The timer is actively timing
DN The preset value has been reached
PRE The target time, in *milliseconds*
ACC The current timer value, in *milliseconds*

Time Base

- Some platforms use a timer time base
- Time base × preset = total time
- AB RSlogix 500: time base can be 0.01s or 1s
  - Base = 0.01, Preset = 500, Time = 5s
- AB RSLogix 5000: time base is *always* 0.001s
  - Base = 0.001, Preset = 500, Time = 0.5s

Timer Variable Warning

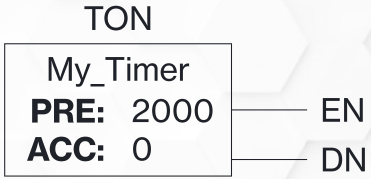
- Timer variables can only be used *once*
- Software will *allow* you to use them more than once, but you will definitely see fun and interesting behavior
- One timer variable per timer instruction

PLC Timers and Counters

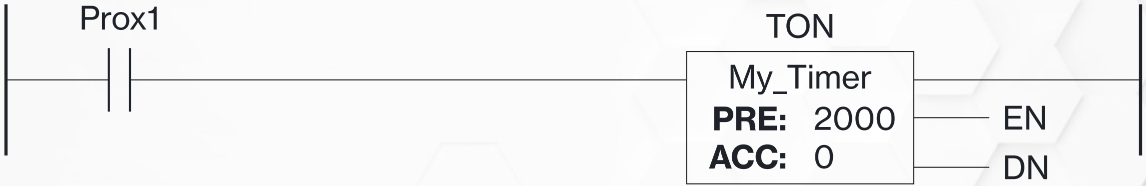
Timer On

Timer On Instruction

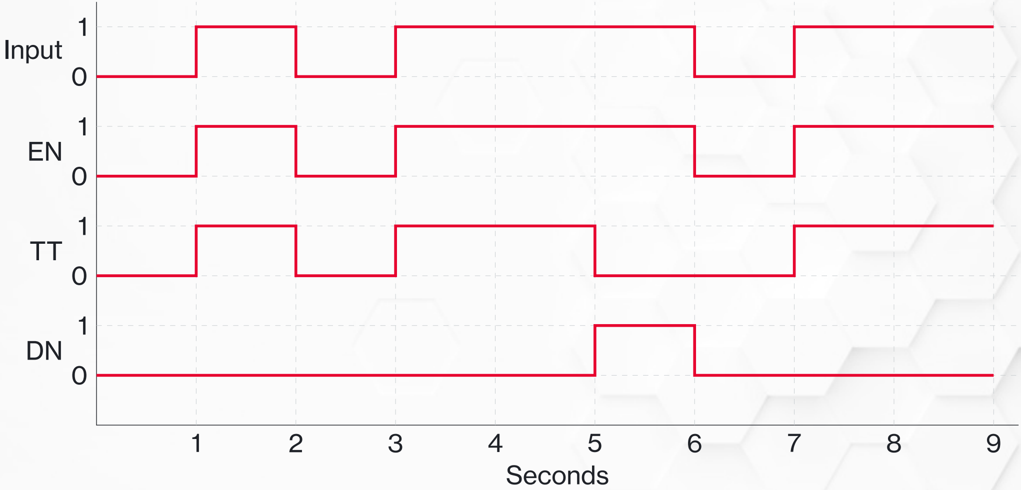
- Starts timing when the preceding logic is true
- Stops timing when the logic goes false
- `ACC` is cleared when the logic goes false



TON Example 1

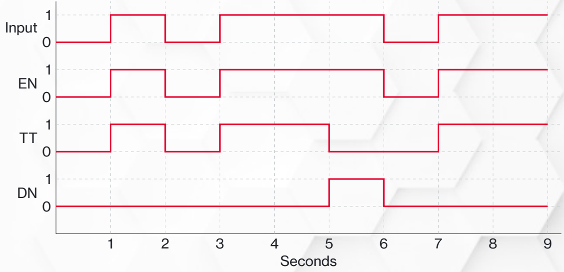


TON Timing Diagram



TON Timing Step-by-Step (1 of 7)

- At 1 second the input goes true
- The logic is true–`EN` turns on
- The timer starts timing–`TT` turns on

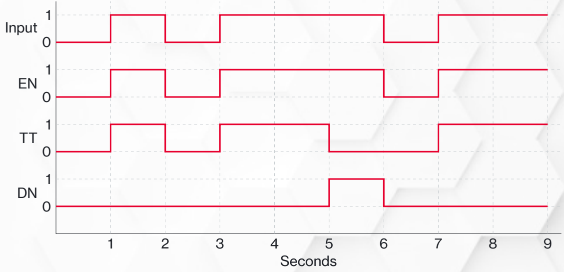


TON Timing Step-by-Step (2 of 7)

While the logic is true `EN` turns on, `TT` turns on, and `ACC` starts counting

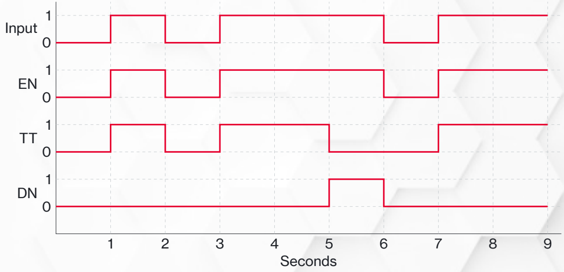
TON Timing Step-by-Step (3 of 7)

- At 2 seconds the input goes false
- The logic is false–`EN` turns off
- The timer stops timing–`TT` turns off
- The `ACC` is automatically cleared
- It wasn't on long enough to reach 2s, `DN` never turns on



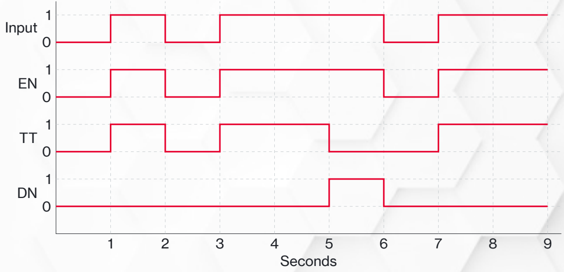
TON Timing Step-by-Step (4 of 7)

- At 3 seconds the input goes true
- The logic is true–`EN` turns on
- The timer starts timing–`TT` turns on
- `ACC` starts counting



TON Timing Step-by-Step (5 of 7)

- At 5 seconds the input is still on
- The logic is true–`EN` stays on
- `ACC` ≥ `PRE`–`DN` turns on
- `ACC` stops counting, holds its value
- It's not actively timing–`TT` turns off

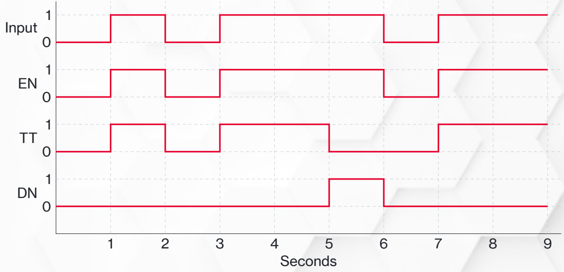


TON Timing Step-by-Step (6 of 7)

At 5 seconds ACC ≥ PRE so DN turns on and TT turns off

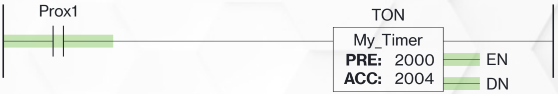
TON Timing Step-by-Step (7 of 7)

- At 6 seconds the input goes false
- The logic is false–`EN` turns off
- The `ACC` is automatically cleared
- `DN` turns off



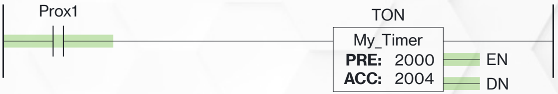
ACC and DN (1 of 3)

- Note the `ACC` value when `DN` turns on–this is normal
- The TON *logic* is triggered by the PLC scan
- The TON *timing* happens in a separate processor thread
- The scan checks in every pass to compare the new `ACC` to `PRE`

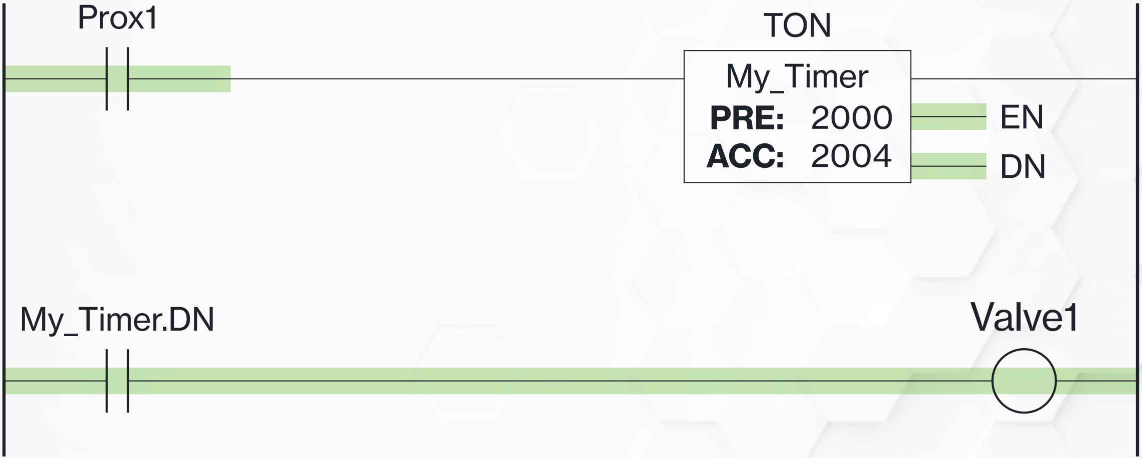


ACC and DN (2 of 3)

- The `ACC` will *always* be slightly bigger than `PRE` when the timer finishes
- You *cannot* do an equal comparison (`ACC=PRE`) because it's *never* true
- That's what `DN` is for



ACC and DN (3 of 3)



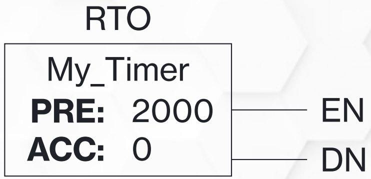
Use `DN` to trigger logic that has to happen when the timer finishes, *not* an equal comparison

PLC Timers and Counters

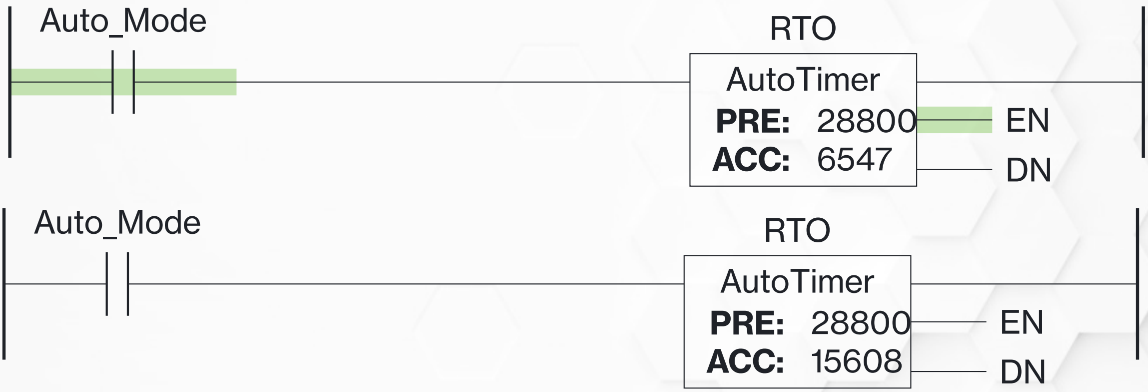
Retentive Timer On

Retentive Timer On (RTO)

- RTO works exactly like TON with *one* difference
- `ACC` *does not* clear when the input goes false
- `ACC` must be explicitly cleared with a RES instruction



RTO Example (1 of 3)

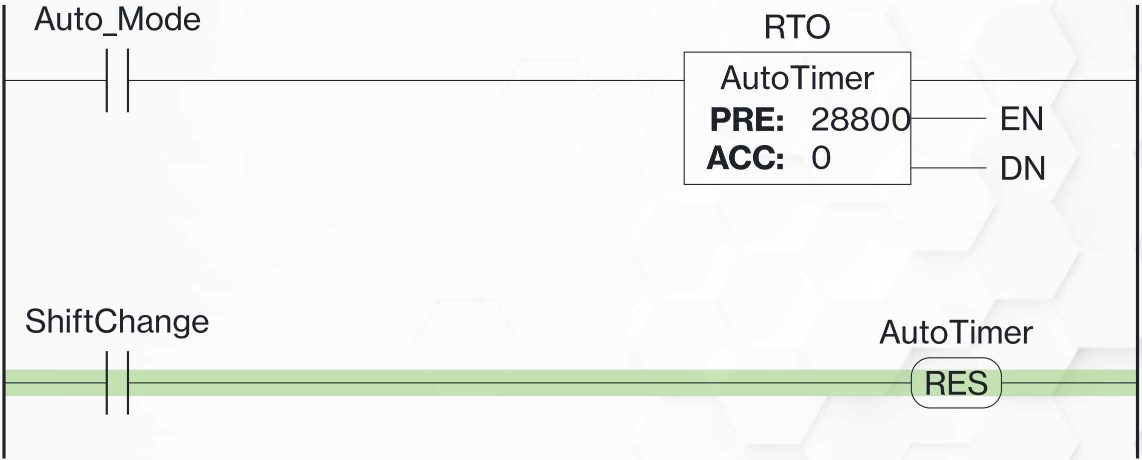


Top: the RTO times as normal when the logic is true. Bottom: When the logic false, the RTO stops timing but retains the `ACC`.

RTO Example (3 of 3)

When PRE is reached DN turns on

RTO Example (3 of 3)



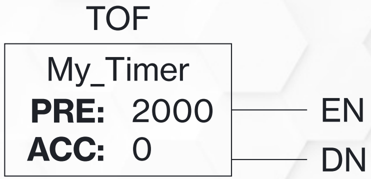
The ACC and DN will *stay on* until a RES instruction is used on the timer

PLC Timers and Counters

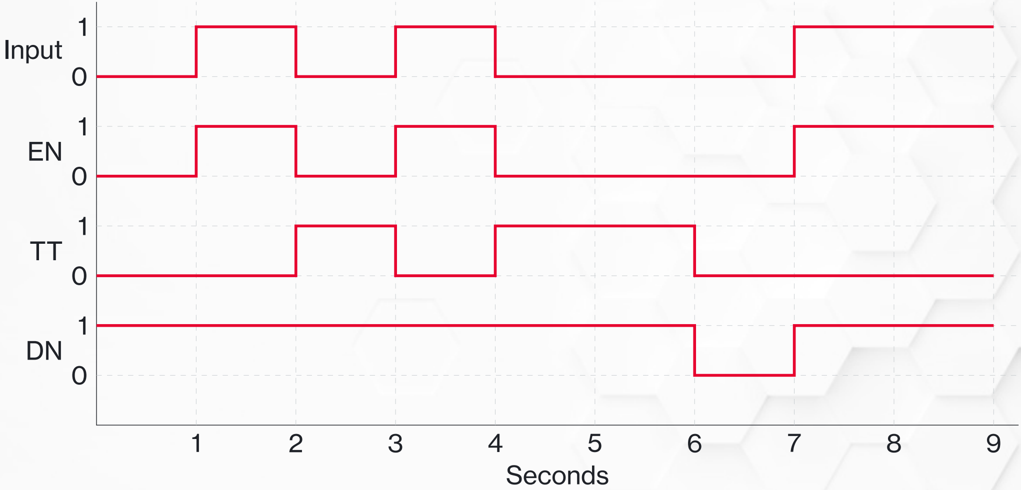
Timer Off

Timer Off Instruction

- Used to keep things running *after* the timer finishes (cooling fans)
- `DN` is on most of the time, it turns *off* when the timer finishes

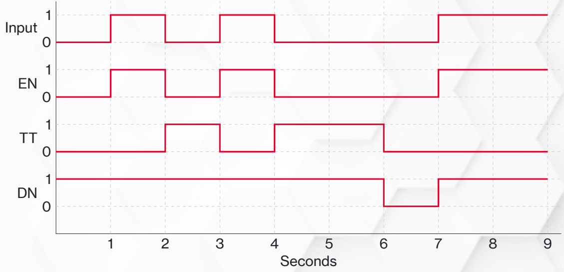


TOF Timing Diagram



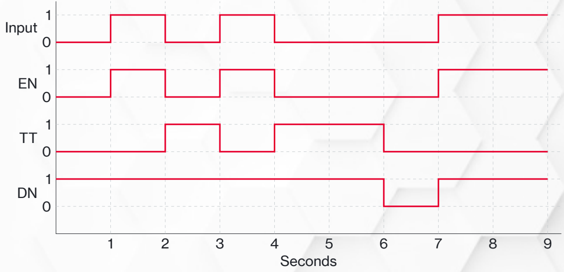
TOF Timing Step-by-Step (1 of 7)

- At 0s the input it off
- `DN` is *already* on



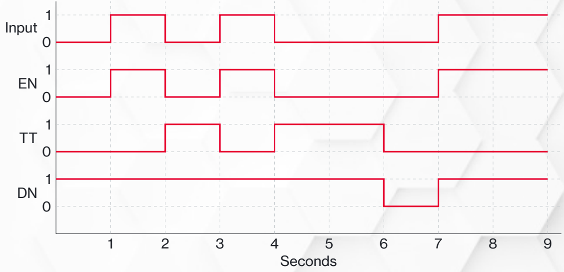
TOF Timing Step-by-Step (2 of 7)

- At 1s the input turns on
- `EN` turns on
- `DN` is *still* on



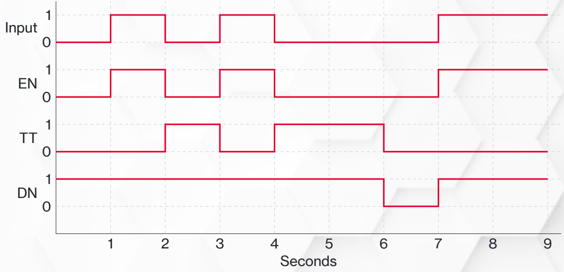
TOF Timing Step-by-Step (3 of 7)

- At 2s the input turns off
- `EN` turns off
- `TT` turns on
- The timer starts timing as soon as the input turns off



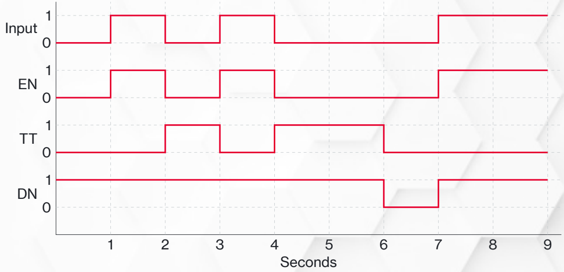
TOF Timing Step-by-Step (4 of 7)

- At 3s the input turns on
- `EN` turns on
- `TT` turns off
- The timer preset hasn't been met, nothing happens with `DN`



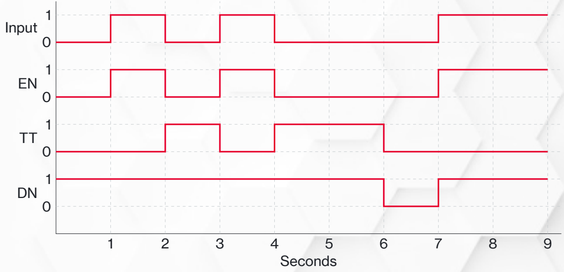
TOF Timing Step-by-Step (5 of 7)

- At 4s the input turns off
- `EN` turns off
- `TT` turns on
- The timer starts timing as soon as the input turns off



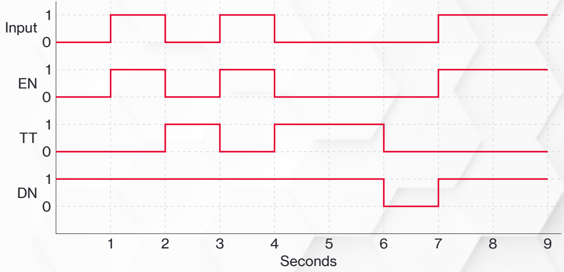
TOF Timing Step-by-Step (6 of 7)

- At 6s we hit the preset
- `TT` turns off–it's not actively timing anymore
- `DN` turns *off*

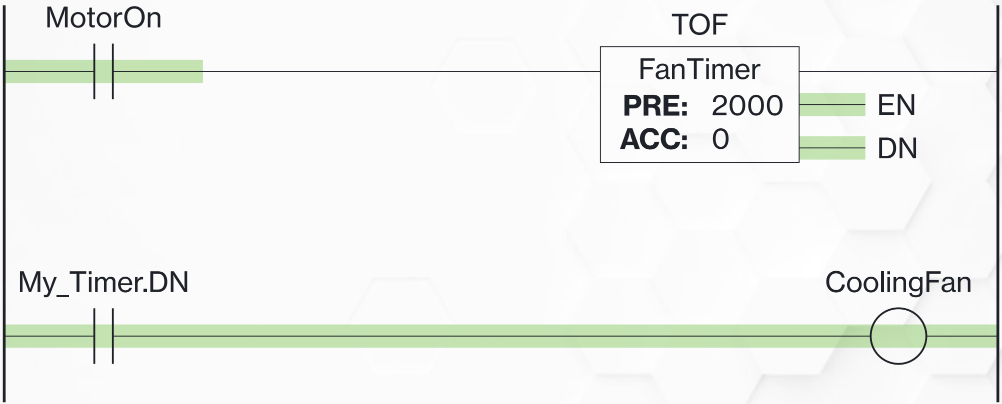


TOF Timing Step-by-Step (7 of 7)

- At 7s the input turns on
- `EN` turns on
- `DN` turns *on*

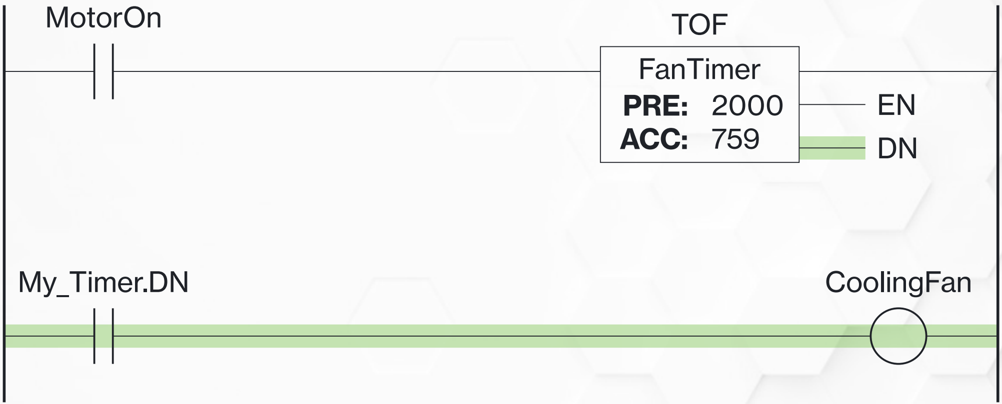


TOF Example (1 of 3)



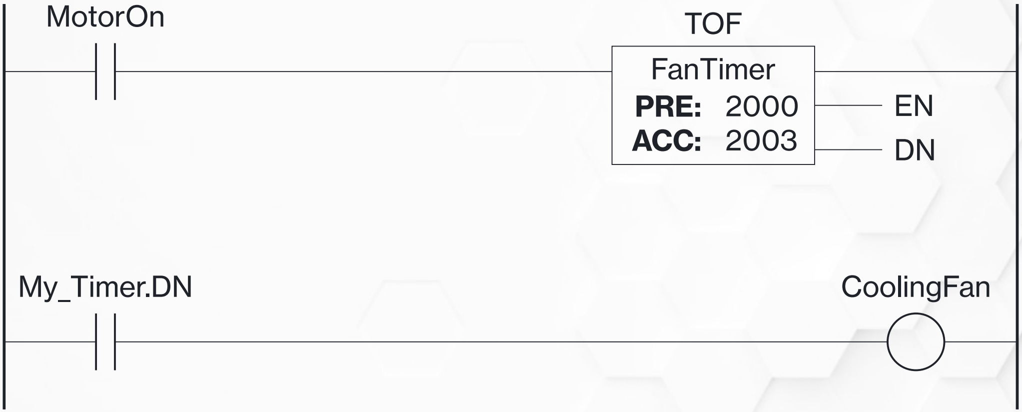
While the input is on, `DN` remains on and timer *is not* timing

TOF Example (2 of 3)



As soon as the input drops out the timer starts timing–`EN` drops, but `DN` stays on

TOF Example (3 of 3)



When we hit `PRE`, `DN` drops out

PLC Timers and Counters

Counters

Counter Variables

CU The logic before the `CTU` instruction is true

CD The logic before the `CTD` instruction is true

DN The preset value has been reached

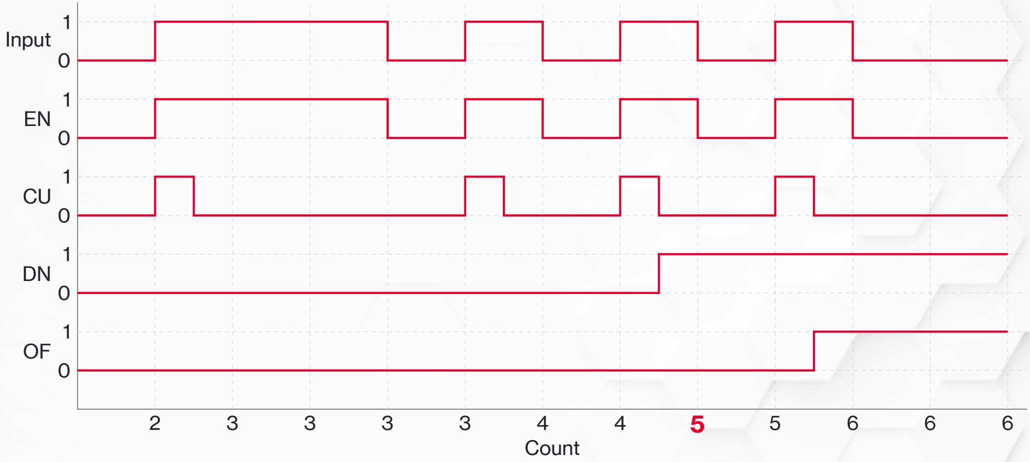
OV The `ACC` has exceeded the DINT limit of 2,147,483,647

UN The `ACC` has exceed the DINT limit of -2,147,483,648

PRE The target count

ACC The current count

CTU Timing Diagram



Advanced PLC Programming Instructions

Data Manupulation Instructions

Copyright Notice